



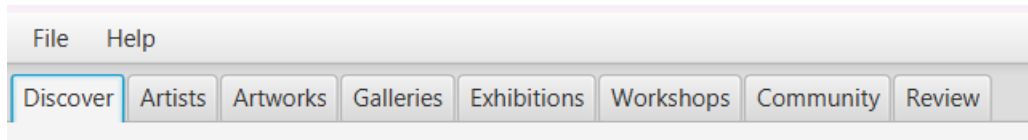
ArtConnect Pro

Rapport final

Lien Github : <https://github.com/melinagrif/ArtConnectPro-App2.git> (branch master)

Étape 1 : Comprendre ArtConnect et définir le périmètre fonctionnel

Les différentes catégories sont :



L'interface est à l'origine organisée autour d'une barre de navigation à 7 onglets (l'onglet "review" a été ajouté lors de la dernière étape) permettant d'accéder aux différents modules de l'application.

Description de chaque écran

Discover (Accueil)

Page d'accueil de l'application qui affiche une synthèse des contenus. Elle présente deux types de cartes, les expositions vedettes (fond bleu) et les workshops à venir (fond vert), cette page donne une vue synthétique de l'activité récente de la plateforme.

Artists

Écran de gestion des artistes avec une barre de recherche (Search artists...) et un tableau à 4 colonnes :

La barre de recherche indique qu'un filtrage dynamique est prévu pour parcourir les artistes.

Artworks

Catalogue des oeuvres d'art avec 5 colonnes (Title, Artist, Type, Price, Status). Les 6 oeuvres actuellement listées sont toutes au statut FOR_SALE, cela suggère l'existence d'autres statuts possibles (ex. SOLD, EXHIBITED).

Galleries

Liste simple des galeries partenaires, affichant le nom, l'adresse et une note sur 5.

Exhibitions

Tableau des expositions avec 4 colonnes : Title, Gallery, Start Date, Theme. Les dates sont au format yyyy-MM-dd, et chaque exposition est associée à une galerie existante, ce qui indique une relation entre les entités Exhibition et Gallery.

Workshops

Tableau des ateliers avec 5 colonnes : Title, Instructor, Date, Price, Level. Le champ Level (Beginner / Intermediate / Advanced) indique l'usage d'une énumération (sous forme de liste) dans le modèle de données.

Community

Espace membres présentant la liste des utilisateurs inscrits avec 3 colonnes : Name, Email, City. Seuls 3 membres fictifs sont présents (Alice Wonderland, Bob Ross, Charlie Brown). Cet écran constitue la base d'un futur système de gestion des utilisateurs avec authentification.

Bilan des entités identifiées

À partir de l'exploration, 6 entités métier ont été identifiées dans l'application :

- Artist - artiste avec nom, ville, email, année de naissance
- Artwork -oeuvre avec titre, type, prix, statut, lien vers un artiste
- Gallery — galerie avec nom, adresse, note
- Exhibition — exposition avec titre, thème, date de début, lien vers une galerie
- Workshop — atelier avec titre, instructeur, date, prix, niveau
- Member (ou User) — membre de la communauté avec nom, email, ville

Diagramme de cas d'utilisation (UML)

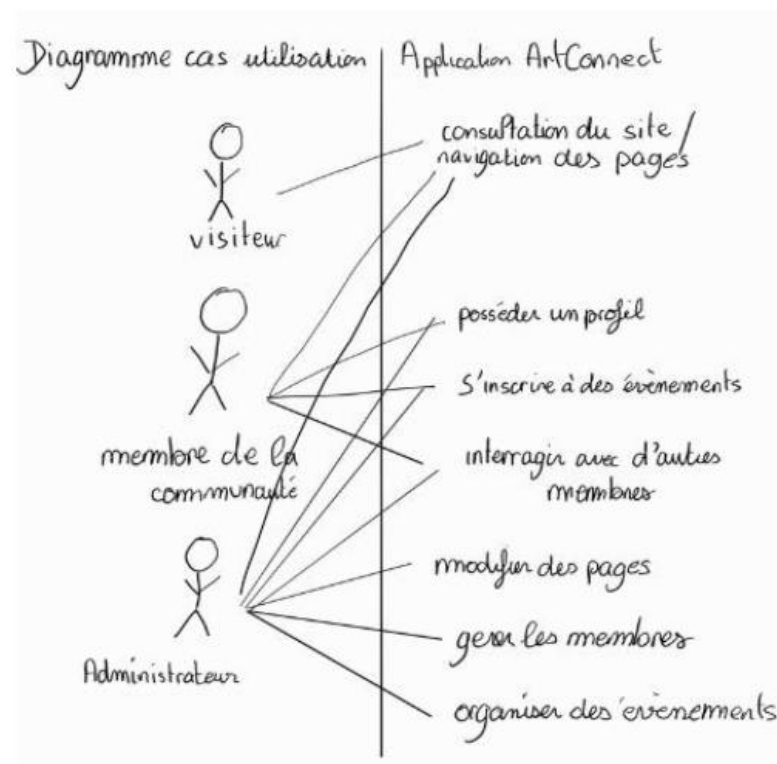


Diagramme de classe



	Workshop
String	title
Date	date
int	duration (minutes)
int	maxParticipants
double	price
Artist	instructor
String	location
String	description
String	level

Étape 2: Modélisation conceptuelle et logique de la base ArtConnect

1. Identifier les entités et relations

Entités identifiées :

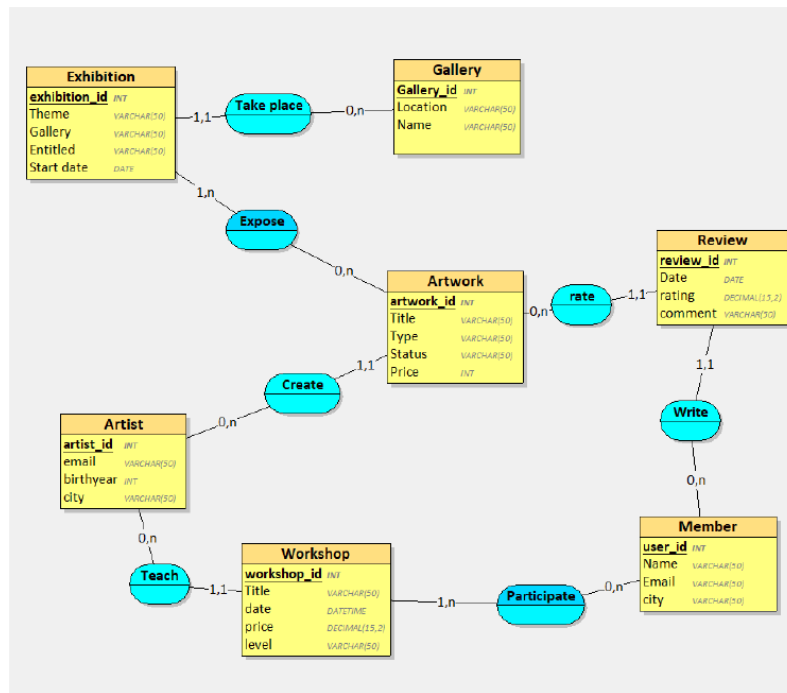
- Artist (Artist_id (PK), Name, City, Email, BirthYear)
- Artwork (Artwork_id (PK), Title, Type, Price, Status, ID_Artist(FK))
- Gallery (Gallery_id(PK), Name, Address, Rating)
- Exhibition (Exhibition_id(PK), Title, Start_Date, Theme, ID_Gallery (FK))
- Workshop (Workshop_id(PK), Title, Instructor, Date, Price, Level), ID_Artist(FK))
- Member(user_id(PK), Name, Email, City)
- Review(review_id (PK), name, email, rating)

Dépendances fonctionnelles :

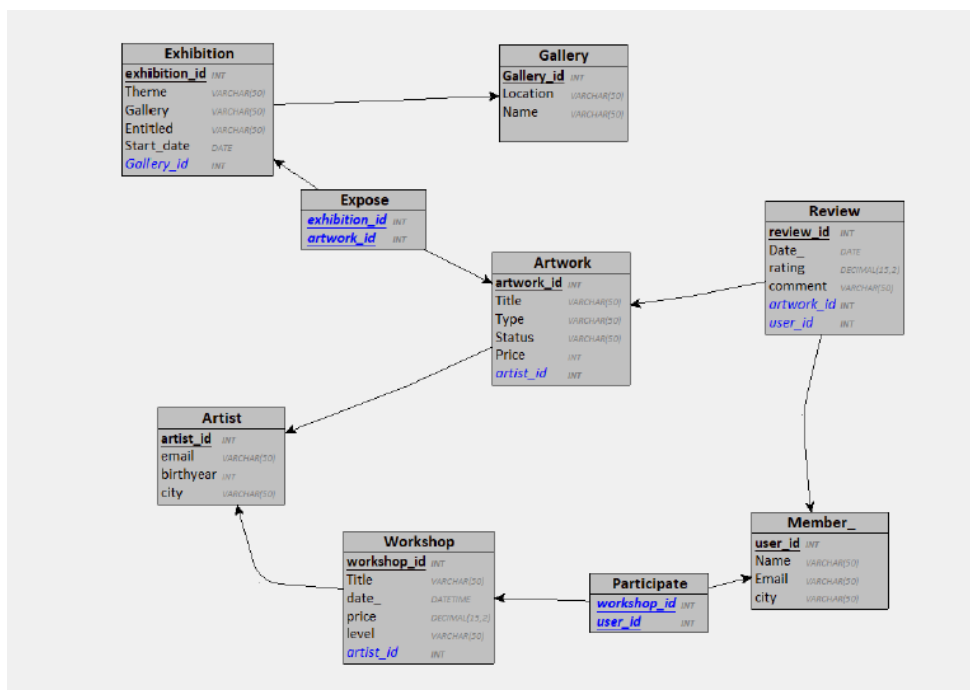
- artist_id → Name, City, Email, Birth_Year
- artwork_id → Title, Artist, Type, Price, Status
- gallery_id → Name, Address, Rating
- exhibition_id → Title, Gallery, Start_Date, Theme
- workshop_id → Title, Instructor, Date, Price, Level
- community_id → Name, Email, City

Avant Normalisation

- Première proposition de MCD

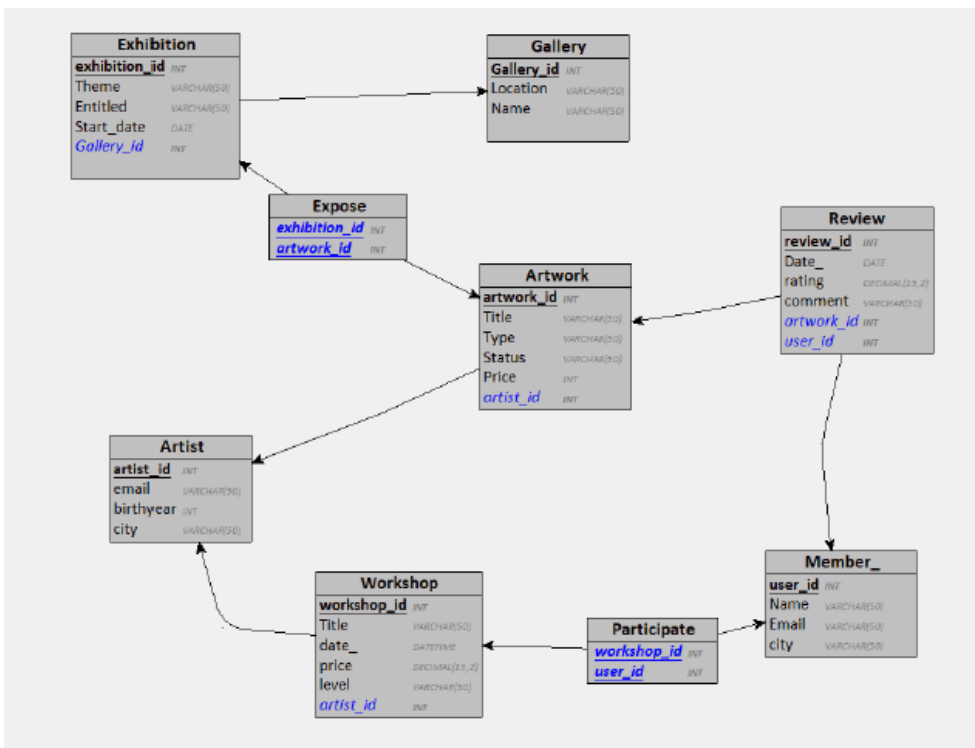
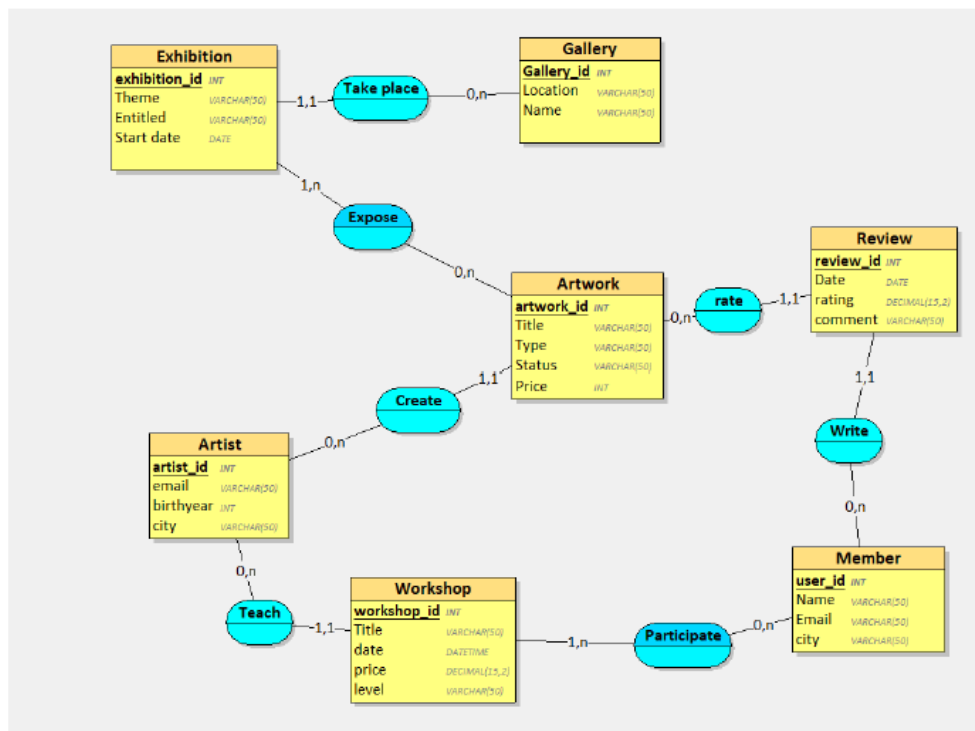


Proposition de MLD :



Normalisation:

Le modèle est en 1NF et 2NF car tous les attributs sont atomiques et les clés sont simples. Il est globalement en 3NF, sauf pour l'attribut Gallery dans Exhibition qui doit être remplacé par une clé étrangère (gallery_id) afin d'éviter une dépendance transitive.



Étape 3 : Implémentation de la Base de Données et Fonctionnalités Avancées

```

6 • CREATE TABLE Artist(
7   artist_id INT,
8   email VARCHAR(50),
9   birthyear INT,
10  city VARCHAR(50),
11  PRIMARY KEY(artist_id)
12 );
13 • CREATE TABLE Artwork(
14  artwork_id INT,
15  Title VARCHAR(50),
16  Type VARCHAR(50),
17  Status VARCHAR(50),
18  Price INT,
19  artist_id INT NOT NULL,
20  PRIMARY KEY(artwork_id),
21  FOREIGN KEY(artist_id) REFERENCES Artist(artist_id)
22 );
23 • CREATE TABLE Gallery(
24  Gallery_id INT,
25  Location VARCHAR(50),
26  Name VARCHAR(50),
27  PRIMARY KEY(Gallery_id)
28 );
29 • CREATE TABLE Workshop(
30  workshop_id INT,
31  Title VARCHAR(50),
32  date_ DATETIME,
33  price DECIMAL(15,2),
34  level VARCHAR(50),
35  artist_id INT NOT NULL,
36  PRIMARY KEY(workshop_id),
37  FOREIGN KEY(artist_id) REFERENCES Artist(artist_id)
38 );
39 • CREATE TABLE Member (
40  user_id INT,
41  Name_member VARCHAR(50),
42  Email VARCHAR(50),
43  city VARCHAR(50),
44  PRIMARY KEY(user_id)
45 );
46 • CREATE TABLE Review(
47  review_id INT,
48  Date_ DATE,
49  rating DECIMAL(15,2),
50  comment VARCHAR(50),
51  artwork_id INT NOT NULL,
52  user_id INT NOT NULL,
53  PRIMARY KEY(review_id),
54  FOREIGN KEY(artwork_id) REFERENCES Artwork(artwork_id),
55  FOREIGN KEY(user_id) REFERENCES Member(user_id)
56 );
57 • CREATE TABLE Exhibition(
58  exhibition_id INT,
59  Theme VARCHAR(50),
60  Entitled VARCHAR(50),
61  Start_date DATE,
62  Gallery_id INT NOT NULL,
63  PRIMARY KEY(exhibition_id),
64  FOREIGN KEY(Gallery_id) REFERENCES Gallery(Gallery_id)
65 );
66 • CREATE TABLE Participate(
67  workshop_id INT,
68  user_id INT,
69  PRIMARY KEY(workshop_id, user_id),
70  FOREIGN KEY(workshop_id) REFERENCES Workshop(workshop_id),
71  FOREIGN KEY(user_id) REFERENCES Member(user_id)
72 );
73 • CREATE TABLE Expose(
74  exhibition_id INT,
75  artwork_id INT,
76  PRIMARY KEY(exhibition_id, artwork_id),
77  FOREIGN KEY(exhibition_id) REFERENCES Exhibition(exhibition_id),
78  FOREIGN KEY(artwork_id) REFERENCES Artwork(artwork_id)
79 );

```

1) Implémentation de la base de données

Avec ChatGPT, on a inséré les données afin d'avoir de vraies données utilisables


```
-- ARTIST
INSERT INTO Artist VALUES
(1, 'monet@gmail.com', 1840, 'Paris'),
(2, 'vangogh@gmail.com', 1853, 'Zundert'),
(3, 'picasso@gmail.com', 1881, 'Malaga');
-- ARTWORK
INSERT INTO Artwork VALUES
(1, 'Water Lilies', 'Painting', 'Available', 500000, 1),
(2, 'Starry Night', 'Painting', 'Sold', 1000000, 2),
(3, 'Guernica', 'Painting', 'Exhibited', 2000000, 3);
-- GALLERY
INSERT INTO Gallery VALUES
(1, 'Paris', 'Louvre'),
(2, 'Amsterdam', 'Van Gogh Museum'),
(3, 'Madrid', 'Reina Sofia');
INSERT INTO Gallery VALUES
(4, 'Paris', 'Musée du Petit Palais');
-- WORKSHOP
INSERT INTO Workshop VALUES
(1, 'Impressionism Basics', '2026-05-10 10:00:00', 150.00, 'Beginner', 1),
(2, 'Post-Impressionism', '2026-06-15 14:00:00', 200.00, 'Intermediate', 2),
(3, 'Cubism Advanced', '2026-07-20 09:00:00', 250.00, 'Advanced', 3);
-- MEMBER
INSERT INTO Member VALUES
(1, 'Alice Dupont', 'alice@gmail.com', 'Paris'),
(2, 'Bob Martin', 'bob@gmail.com', 'Lyon'),
(3, 'Charlie Durand', 'charlie@gmail.com', 'Marseille');

-- REVIEW
INSERT INTO Review VALUES
(1, '2026-03-01', 4.5, 'Magnifique', 1, 1),
(2, '2026-03-05', 5.0, 'Incroyable', 2, 2),
(3, '2026-03-10', 4.0, 'Très bien', 3, 3);

-- EXHIBITION
INSERT INTO Exhibition VALUES
(1, 'Impressionism', 'Light & Nature', '2026-04-01', 1),
(2, 'Post-Impressionism', 'Colors of Emotion', '2026-04-15', 2),
(3, 'Cubism', 'Shapes & Forms', '2026-05-01', 3);

INSERT INTO Exhibition VALUES
(4, 'Realism', 'True Nature', '2027-01-03', 3);

-- PARTICIPATE
INSERT INTO Participate VALUES
(1, 1),
(2, 2),
(3, 3);

-- EXPOSE
INSERT INTO Expose VALUES
(1, 1),
(2, 2),
(3, 3);
```

2) Vues, index et droits d'accès

Vues:

- **CREATE VIEW catalogue AS**
SELECT exhibition_id, Theme, Entitled, Start_date, Location, Name
FROM Exhibition **JOIN** Gallery **ON** Exhibition.Gallery_id = Gallery.Gallery_id;
- **SELECT * FROM catalogue;**

-- On peut créer une vue d'une oeuvre avec toutes les infos sur les avis
- **CREATE VIEW oeuvre_notes AS**
SELECT Title, date_, rating, comment **FROM** Artwork **INNER JOIN** Review
ON Artwork.artwork_id = Review.artwork_id;
- **SELECT * from oeuvre_notes;**
--
- **CREATE VIEW v_exhibition_catalog AS**
SELECT
e.exhibition_id,
e.Entitled **AS** exposition,
e.Theme,
e.Start_date,
g.name **AS** galerie,
g.Location,
a.artwork_id,
a.Title **AS** oeuvre,
a.Type,
ar.artist_id
FROM Exhibition e
JOIN Gallery g **ON** e.Gallery_id = g.Gallery_id
JOIN Expose ex **ON** e.exhibition_id = ex.exhibition_id
JOIN Artwork a **ON** ex.artwork_id = a.artwork_id
JOIN Artist ar **ON** a.artist_id = ar.artist_id;

-- Vue permettant d'afficher les informations publiques de tous les profils

CREATE VIEW v_member_public AS

SELECT

user_id,

Name_member,

city

FROM Member;

- On peut créer un catalogue des expositions.
- On peut créer une vue d'une oeuvre avec toutes les infos sur les avis.
- Simplification : catalogue complet d'une exposition.
- Vue permettant d'afficher les informations publiques de tous les profils

Index:

```
-- INDEX 1 - Recherche fréquente : œuvres par artiste
-- Justification : la jointure Artwork et Artist est omniprésente ;
CREATE INDEX idx_artwork_artist ON Artwork(artist_id);

-- INDEX 2 - Filtrage par statut d'œuvre
-- Justification : les requêtes "WHERE Status = 'Disponible'" sont
-- très courantes sur la page catalogue publique.
CREATE INDEX idx_artwork_status ON Artwork(Status);

-- INDEX 3 - Recherche d'ateliers par date
-- Justification : l'affichage "prochains ateliers" trie/filtre sur date_.
CREATE INDEX idx_workshop_date ON Workshop(date_);
```

Triggers/déclencheurs :

TRIGGER 1 – La note d'un avis doit être entre 0 et 5

```
CREATE TRIGGER trg_review_rating
BEFORE INSERT ON Review
FOR EACH ROW
BEGIN
IF NEW.rating < 0 OR NEW.rating > 5 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'La note doit être entre 0 et 5.';
END IF;
END$$
```

TRIGGER 2 – Le prix d'une œuvre doit être positif

```
CREATE TRIGGER trg_artwork_price
BEFORE INSERT ON Artwork
FOR EACH ROW
BEGIN
IF NEW.Price < 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Le prix d'une œuvre ne peut pas être négatif.';
END IF;
END$$
```

TRIGGER 3 – Le prix d'un workshop (atelier) doit être positif

```
-- TRIGGER 3 – Le prix d'un atelier doit être positif
CREATE TRIGGER trg_workshop_price
BEFORE INSERT ON Workshop
FOR EACH ROW
BEGIN
  IF NEW.price < 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Le prix d\'un atelier ne peut pas être négatif.';
  END IF;
END$$
```

PROCEDURE :

1- Procédure qui permet d'obtenir le prix moyen de vente d'un artiste

```
CREATE PROCEDURE GetStatistiquesArtiste(
  IN p_artist_id INT
)
BEGIN
  SELECT
    A.artist_id,
    A.email,
    COUNT(W.artwork_id) AS Nombre_Oeuvres,
    AVG(W.Price) AS Prix_Moyen
  FROM Artist A
  LEFT JOIN Artwork W ON A.artist_id = W.artist_id
  WHERE A.artist_id = p_artist_id
  GROUP BY A.artist_id;
END //
```

2- Procédure analysant si l'artiste d'une œuvre est bien répertoriée

```
CREATE PROCEDURE AjouterArtwork(
  IN p_id INT,
  IN p_titre VARCHAR(50),
  IN p_type VARCHAR(50),
  IN p_status VARCHAR(50),
  IN p_prix INT,
  IN p_artist_id INT
)
BEGIN
  IF EXISTS (SELECT 1 FROM Artist WHERE artist_id = p_artist_id) THEN
    INSERT INTO Artwork (artwork_id, Title, Type, Status, Price, artist_id)
    VALUES (p_id, p_titre, p_type, p_status, p_prix, p_artist_id);
    SELECT 'L\'œuvre a été ajoutée avec succès.' AS Message;
  ELSE
    SELECT 'Erreur : Cet artiste n'existe pas.' AS Message;
  END IF;
END //
```

TRANSACTION :

Elle garantit qu'un membre est soit complètement inscrit (avec son profil, son inscription à l'atelier et son avis) ou soit pas du tout.

```
START TRANSACTION;

INSERT INTO Member(user_id, Name_member, Email, city)
VALUES (p_user_id, p_name, p_email, p_city);

INSERT INTO Participate(workshop_id, user_id)
VALUES (p_workshop_id, p_user_id);

INSERT INTO Review(review_id, Date_, rating, comment, artwork_id, user_id)
VALUES (
    (SELECT IFNULL(MAX(review_id), 0) + 1 FROM Review r),
    CURDATE(),
    p_rating,
    p_comment,
    p_artwork_id,
    p_user_id
);

COMMIT;
END$$
```

Étape 4 : Intégration de la base dans l'application Java ArtConnect

1. DAO

- Complétions des interfaces des DAO (ici Artist)

```
public interface ArtistDao { 4 usages 1 imple
    List<Artist> findAll(); 1 implementation

    void save(Artist artist); 1 usage 1 imple

    void update(Artist artist); 1 usage 1 im

    void delete(String artistName); 1 usage

    List<Artist> findByCity(String city);

}
```

- Implémentation des DAO :

findAll() : la méthode la plus importante, elle montre le pattern JDBC complet

```
public class JdbcArtistDao implements ArtistDao { 2 usages

    @Override
    public List<Artist> findAll() {
        List<Artist> artists = new ArrayList<>();
        String sql = "SELECT artist_id, name, email, birthyear, city FROM artist";

        try (Connection conn = DatabaseConfig.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                artists.add(mapRow(rs));
            }
        } catch (SQLException e) {
            throw new RuntimeException("Erreur lors de la récupération des artistes", e);
        }

        return artists;
    }
}
```

save() : montre l'insertion

```
@Override 1 usage
public void save(Artist artist) {
    String sql = "INSERT INTO artist (name, email, birthyear, city) VALUES (?, ?, ?, ?)";

    try (Connection conn = DatabaseConfig.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, artist.getName());
        stmt.setString(2, artist.getEmail());
        stmt.setObject(3, artist.getBirthYear());
        stmt.setString(4, artist.getCity());
        stmt.executeUpdate();
    } catch (SQLException e) {
        throw new RuntimeException("Erreur lors de l'insertion de l'artiste : " + artist.getName(), e);
    }
}
```

mapRow() : montre comment on map les colonnes SQL vers l'objet Java

```
private Artist mapRow(ResultSet rs) throws SQLException {
    Artist a = new Artist();
    a.setArtist_id(rs.getString(columnLabel: "artist_id"));
    a.setName(rs.getString(columnLabel: "name"));
    a.setEmail(rs.getString(columnLabel: "email"));
    a.setBirthYear(rs.getInt(columnLabel: "birthyear"));
    a.setCity(rs.getString(columnLabel: "city"));
    return a;
}
```

2. SERVICES

```
public class ServiceProvider { 16 usages

    private static final ArtistService  artistService  = new JdbcArtistService(); 1 usage
    private static final ArtworkService artworkService = new JdbcArtworkService(); 1 usage
    private static final GalleryService galleryService = new JdbcGalleryService(); 1 usage
    private static final WorkshopService workshopService = new JdbcWorkshopService(); 1 usage
    private static final CommunityService communityService = new JdbcCommunityService(); 1 usage

    > public static ArtistService getArtistService() { return artistService; }
    > public static ArtworkService getArtworkService() { return artworkService; }
    > public static GalleryService getGalleryService() { return galleryService; }
    > public static WorkshopService getWorkshopService() { return workshopService; }
    > public static CommunityService getCommunityService() { return communityService; }
}
```

La couche service fait le lien entre l'interface utilisateur et les DAO. Par exemple, JdbcArtistService reçoit un JdbcArtistDao en paramètre et délègue les opérations à ce dernier. Les services sont instanciés une seule fois et partagés dans toute l'application.

3. Connexion à la base

```
public class DatabaseConfig { 30 usages

    private static final String URL      = "JDBC:mysql://127.0.0.1:3306/Artconnect"; 1 usage
    private static final String USER     = "root"; // à adapter 1 usage
    private static final String PASSWORD = "root"; // à adapter 1 usage

    static {
        try {
            Class.forName( className: "com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new ExceptionInInitializerError( s: "Driver MySQL introuvable : " + e.getMessage());
        }
    }

    public static Connection getConnection() throws SQLException { 23 usages
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    private DatabaseConfig() {} no usages
}
```

6. Etap

4. Adaptation de l'UI

Name	City	Email	Birth Year
	Paris	monet@gmail.com	1840
	Zundert	vangogh@gmail.com	1853
	Malaga	picasso@gmail.com	1881
Anastasia Kreslina	Riga	NK@gmail.ru	1990

```
INSERT INTO Artist VALUES(4, 'Anastasia Kreslina', 'NK@gmail.ru', 1990, 'Riga');
```

5. Diagramme



6. Ajout d'un onglet supplémentaire

Afin de rendre l'expérience client plus complète, nous avons décidé d'ajouter l'onglet "review" à l'application. Elle permet aux utilisateurs de laisser des commentaires et une note sur chaque œuvre.

Discover

Artists

Artworks

Galleries

Exhibitions

Workshops

Community

Review

Filter by member

Search

Reset

Member	Artwork	Rating	Comment	Date	
Alice Dupont	Water Lilies	★★★★.	Magnifique	2026-03-01	
Bob Martin	Starry Night	★★★★★	Incroyable	2026-03-05	
Charlie Durand	Guernica	★★★★.	Très bien	2026-03-10	
Bob Martin	Guernica	★★★..	Interessant, je reviendrai !	2026-05-10	

l